SEPTEMBER 29, 2022 / #BEGINNERS GUIDE

CSS Selectors – Cheat Sheet for Class, Name, Child Selector List

Dionysia Lemonaki

CSS selectors target and select the HTML elements you want to style.

Specifically, CSS selectors allow you to select multiple elements at once.

They are helpful when you want to apply the same styles to more than one HTML element, because you will not repeat yourself by writing the same lines of code for different elements.

CSS selectors are also helpful when you want to make a change - you only need to make the change in one place, which saves you a lot of time.

CSS selectors are among the first things you need to learn when you first start writing CSS code.

And there are many selectors available to choose from, along with several different ways to use them - more than you may realize.

With that said, there is no need to worry - you do not have to memorize everything.

This cheat sheet covers the most commonly used selectors you need to know when starting out. Bookmark it so you can come back to it whenever you need a quick reminder when you are working on your next web design project.

Here is what we will cover:

Simple CSS Selectors

Selectors allow you to target and select specific parts of your document for styling purposes.

Simple selectors directly select one or more elements:

- By using the universal selector, `*`.
- Based on the name/type of the element.
- Based on the class value of the element.
- Based on the ID value of the element.

By learning how the most simple selectors work, you can understand how to use the more complex ones.

The simple selectors will most often be the ones you will use the most and the ones you will be the most familiar with if you have some experience writing CSS code.

CSS Universal Selector

The universal selector, also known as a wildcard, selects everything - every single element in the document.

To use the universal selector, use the asterisk character, `*`.

```
* {
    property: value;
}
```

You can use the universal selector to reset the browser's default padding and margin to zero at the top of the file before you add any other styles:

```
* {
    padding: 0;
    margin:  0;
}
```

CSS Type Selector

The CSS type selector selects all HTML elements of the specified type.

To use it, mention the name of the HTML element.

For example, if you wanted to apply a style to every single paragraph in the HTML document, you would specify the `p` element:

```
p {
    property: value;
}
```

The code above matches and selects *all* `p` elements within the document and styles them.

CSS Class Selector

The class selector matches and selects HTML elements based on the value of their given class. Specifically, it selects every single element in the document with that specific class name.

With the class selector, you can select multiple elements at once and style them the same way without copying and pasting the same styles for each one separately.

Classes are reusable, making them a good option for practicing DRY development. DRY is a programming principle and is short for 'Don't Repeat Yourself'. As the name suggests, the aim is to avoid writing repetitive code whenever possible.

To select elements with the class selector, use the dot character, `.`, followed by the name of the class.

```
.my_class {
    property: value;
}
```

In the code above, elements with a class of `my_class` are selected and styled accordingly.

CSS ID Selector

The ID selector selects an HTML element based on the value of its ID attribute.

Keep in mind that the ID of an element should be unique in a document, meaning there should only be one HTML element with that given ID value. You cannot use the same ID value on a different element besides that one.

To select an element with a specific ID, use the hash character, `#`, followed by the name of the ID value:

```
#my_id {
    property: value;
}
```

The code above will match only the unique element with the ID value of `my_id`.

It's worth mentioning that it is best to try and limit the use of this selector and opt for using the class selector instead. Applying styles

using the ID selector is not ideal because the styles are not reusable.

CSS Attribute Selectors

Many HTML elements have attributes.

HTML attributes:

- Provide additional information about HTML elements.
- Are always specified in the start (or opening) tag.
- Usually come in name/value pairs such as `name="value"`.
- The `value` in a name/value pair should be enclosed in quotation marks.

One of the most popular HTML attributes you may have come across is the `href` attribute, which is added to the opening `<a>` tag and specifies the URL the `<a>` tag links to:

```
<a href="https://www.freecodecamp.org/">The best place to learn to code
```

The value of the `href`, `https://www.freecodecamp.org/`, is the URL the user will be taken to when they click on the link text, `The best place to learn to code for free!`.

The attribute selector matches and selects HTML elements based on the presence of an attribute or a specific attribute value.

There are different types of attribute selectors.

Below are some of the most common attribute selectors.

The `[attribute]` Selector

To use the attribute selector, use a pair of square brackets, `[]`, to select the attribute you want.

The general syntax for attribute selectors is the following:

```
element[attribute]
```

This selector selects an element if the given attribute exists.

In the following example, elements that have the attribute `attr` present are selected, regardless of the specific value of `attr`:

```
a[attr] {
    property: value;
}
```

In the example above, `a` elements with the `attr` attribute name are selected, regardless of the value of `attr`.

With that said, you can be more specific with your styling.

The `[attribute="value"]` Selector

You can specify the value of the attribute using the following syntax:

```
element[attribute="value"]
```

So, if you want to style `a` elements with an `attr` attribute that has an **exact** value of `1`, you would do the following:

```
a[attr="1"] {
    property: value;
}
```

This code above matches `a` elements where the `attr` attribute name has an exact value of `1`.

The `[attribute^="value"]` Selector

You could also specify that the value of the attribute **starts** with a specific character using the following syntax:

```
element[attribute^="value"]
```

For example, if you wanted to select and style any `a` elements that have an `attr` attribute with a value that starts with `www`, you would do the following:

```
a[attr^="www"] {
    property: value;
}
```

The code above selects any `a` elements where the `attr` attribute name has a value that starts with `www`.

The `[attribute$="value"]` Selector

You could also specify that the value of the attribute **ends** with a specific character using the following syntax:

```
element[attribute$="value"]
```

For example, if you wanted to select `a` elements that have an `attr` attribute name with a value that ends with `.com`, you would do the following:

```
a[attr$=".com"] {
    property: value;
}
```

The `[attribute*="value"]` Selector

You can also specify that the attribute value contains a specific substring - this selector is known as the Attribute Contains Substring Selector and has the following syntax:

```
element[attribute*="value"]
```

In this case, the string `value` needs to be present in the attribute's value followed by any number of other characters - `value` doesn't need to be a whole word.

For example, if you wanted to select `a` elements that have an `attr` attribute with a value that contains the string `free`, you would do the following:

```
a[attr*="free"] {
    property:value;
}
```

The code above selects `a` elements with an `attr` attribute name when the string `free` is present in the attribute's value - even as a substring (a substring is a word inside another word).

As long as the attribute's value contains `free`, then the HTML element is selected - this could match an `attr` attribute with a value of `free`, `freeCodeCamp`, or `freediving`, for example.

The `[attribute~="value"]` Selector

You can also specify that the selector matches an attribute value that contains a whole word using the following syntax:

```
element[attribute~= "value"]
```

In this case, the string `value` needs to be a whole word.

For example, if you wanted to select `a` elements that have an `attr` attribute name with a value that contains the word `free`, you would do the following:

```
a[attr~= "free"] {
    property: value;
}
```

The code above would match an `attr` attribute with a value of `free` that contains different kinds of whitespace.

The code wouldn't select elements with an `attr` value of `freeCodeCamp` or `freediving` like you saw in an earlier example because `free` needs to be a whole word on its own - not a substring.

Grouping CSS Selector

With the grouping selector, you can target and style more than one element at once.

To use the grouping selector, use a comma, `,` to group and separate the different elements you want to select.

For example, here is how you would target multiple elements such as `div` s, `p` s, and `span` s all at once and apply the same styles to each of them:

```
div, p, span {
    property: value;
}
```

The code above matches all `div` , `p` , and `span` elements on the page, and those three elements will share the same styling.

CSS Combinators

Combinators allow you to combine two elements based on the relationship between the elements and their location in the document.

Essentially, you can combine two simple selectors in a way that explains the relationship between those CSS selectors. Combinators are a type of selector that specifies and describes the relationship between the two selectors.

There are four types of combinators:

- The descendant combinator.
- The direct child combinator.
- The general sibling combinator.
- The adjacent sibling combinator

Descendant Combinator

As the name implies, the descendant combinator selects only the descendants of the specified element.

Essentially, you first mention the parent element, leave a space, and then mention the descendant of the first element, which is the child element of the parent. The child element is an element inside the parent element.

Let's take the following as an example:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta http-equiv="X-UA-Compatible" content="IE=edge">
  <meta name="viewport" content="width=device-width, initial-scale=1.0"
  <link rel="stylesheet" href="main.css">
  <title>Document</title>
</head>
<body>
  <div>
    <h2>I am level 2 heading</h2>
    <p>I am a paragraph inside a div</p>
```

```
    <span>I am a span</span>
    <p>I am a paragraph inside a div</p>
  </div>
  <p>I am a paragraph outside a div</p>
</body>
</html>
```

In the example above, the `div` is the parent, and the `h2`, `span`, and two `p`s are the child elements because they are inside the `div`. There is also a paragraph outside the `div`.

If you wanted to style only the paragraphs that are inside the `div`, here is what you would do:

```
div p {
    property: style;
}
```

In the example above, only the two paragraphs inside the `div` that have the text `I am a paragraph inside a div` get styled. The other two child elements and the paragraph outside the `div` with the text `I am a paragraph outside a div` do not get styled.

Direct Child Combinator

The direct child combinator, also known as the direct descendant, selects only the direct children of the parent.

To use the direct child combinator, specify the parent element, then add the `>` character followed by the direct children of the parent element you want to select.

Let's take the following as an example:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta http-equiv="X-UA-Compatible" content="IE=edge">
  <meta name="viewport" content="width=device-width, initial-scale=1.0
  <link rel="stylesheet" href="main.css">
  <title>Document</title>
</head>
<body>
  <div>
    <a href="#">I am a link</a>
    <a href="#">I am a link</a>
    <p><a href="#">I am a link inside a paragraph</a></p>
  </div>
</body>
</html>
```

There is a `div` which is the parent element.

Inside the parent element, there are two `a` elements which are the direct children of the `div`.

There is also another `a` element inside a `p` element. The `p` element is a child of the `div`, but the `a` element inside the paragraph is *not* a direct child of the `div`.

So, to access only the `a` elements that are direct children of `div`, you would do the following:

```
div > a  {
    property: value;
}
```

The code above matches the `a` elements directly nested inside the `div` element and are immediate children.

General Sibling Combinator

The general sibling combinator selects siblings.

You can specify the first element and a second one that comes after the first one. The second element doesn't need to come right after the first one.

To use the general sibling combinator, specify the first element, then use the `~` character followed by the second element that needs to follow the first one.

Let's take the following as an example:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta http-equiv="X-UA-Compatible" content="IE=edge">
  <meta name="viewport" content="width=device-width, initial-scale=1.0'
  <link rel="stylesheet" href="main.css">
  <title>Document</title>
</head>
<body>
  <div>
    <p>I am paragraph inside a div</p>
  </div>
  <p>I am a paragraph outside a div</p>
  <h3>I am a level three heading</h3>
  <p>I am a paragraph outside a div</p>
</body>
</html>
```

The `div` has a `p` element nested inside it. That specific `p` element is a child of `div`.

There are also two paragraphs with the text `I am a paragraph outside a div` and an `h3` element. All those three elements are siblings of the `div` element.

So, to select the `p` elements that are siblings of the `div` element, you would do the following:

```
div ~ p {
    property: value;
}
```

The code above styles both `p` elements that come after the `div`.

It styles even the `p` element that does not come directly after the `div` element, such as the `p` that follows the `h3` element. It does so because it still comes after `div`.

Adjacent Sibling Combinator

The adjacent sibling combinator is more specific than the general sibling combinator.

This selector matches only the *immediate* siblings. Immediate siblings are the siblings that come right after the first element.

To use the adjacent sibling combinator, specify the first element, then add the `+` character followed by the element you want to select that immediately follows the first element.

Let's revisit the example from the previous section:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta http-equiv="X-UA-Compatible" content="IE=edge">
  <meta name="viewport" content="width=device-width, initial-scale=1.0'
  <link rel="stylesheet" href="main.css">
  <title>Document</title>
</head>
<body>
  <div>
    <p>I am paragraph inside a div</p>
  </div>
  <p>I am a paragraph outside a div</p>
  <h3>I am a level three heading</h3>
  <p>I am a paragraph outside a div</p>
</body>
</html>
```

Although the `p` element that follows the `h3` element is a sibling of the `div` element, it is not a *direct* sibling like the `p` element that comes before the `h3` .

So, to target only the `p` element that comes directly after the `div` , you would do the following:

```
div + p {
    property: value;
}
```

Pseudo-Class Selectors

Pseudo-class selectors select elements that are in a specific state.

Some examples of the state the element can be in are:

- The element is being hovered over by the mouse pointer.
- The element is the first of its type.
- The link has been visited before from that specific browser.
- The link has *not* been visited before from that specific browser.
- The checkbox/radio button has been checked.

Pseudo-class selectors start with a colon, `:`, followed by a keyword that reflects the state of the specified element.

The general syntax looks something like the following:

```
element:pseudo-class-name {
    property: value;
}
```

Pseudo-Class Selectors for Links

There are selectors based on link state information.

The `:link` selector applies styling when the element has not been visited before:

```
a:link {
    property: value;
}
```

The `:visited` selector applies when the element has been visited before in the current browser:

```
a:visited {
    property: value;
}
```

The `:hover` selector applies when the mouse pointer hovers over an element:

```
a:hover {
    property: value;
}
```

The `:focus` selector applies when a user has tabbed onto an element:

```
a:focus {
    property:value;
}
```

The `:active` selector applies when the element is selected after being clicked on and after holding down a mouse button:

```
a:active {
    property: value;
}
```

Pseudo-Class Selectors for Inputs

The `:focus` selector you saw earlier for links is used for inputs as well:

```
input:focus {
    property: value;
}
```

The `:required` selector selects inputs that are required. Inputs that are required have the `required` attribute.

```
input:required {
    property: value;
}
```

The `:checked` selector selects checkboxes or radio buttons that have been checked:

```
input:checked {
    property:value;
}
```

The `:disabled` selector selects inputs that are disabled. Disabled inputs have the `disabled` attribute. Many browsers style disabled inputs with a faded-out gray color by default:

```
input:disabled {
    property:value;
}
```

Pseudo-Class Selectors for Position

The first child selector, `:first-child`, selects the first element, which will be the first child inside the parent container.

For example, here is how you would select an `a` element when it is the first child in the parent container:

```
a:first-child {
    property: value;
}
```

The last child selector, `:last-child`, selects the last element, which will be the last child inside the parent container.

Here is how you would select an `a` element when it is the last child in the parent container:

```
a:last-child {
    property: value;
}
```

The `:nth-child()` selector selects a child element inside a container based on its position in a group of siblings.

It takes an integer as an argument and selects an element based on the given value. The general syntax for the selector looks something like this:

```
a:nth-child(n) {
    property: value;
}
```

The `:nth-child()` selector is helpful when you want to select elements based on an expression, such as selecting even or odd elements:

```
a:nth-child(even) {
    property: value;
}
```

The first of type selector, `:first-of-type`, selects elements that are the first of that specific type in the parent container.

For example, here is how you would select the first `p` inside a `div`:

```
p:first-of-type {
    property: value;
}
```

The last of type selector, `:last-of-type`, selects elements that are the last of that specific type in the parent container.

For example, here is how you would select the last `p` inside a `div`:

```
p:last-of-type {
    property: value;
}
```

Pseudo-Element Selectors

Pseudo-element selectors are used for styling a specific part of an element - you can use them to insert new content or change the look of a specific section of the content.

For example, you can use a pseudo-element to style the first letter or the first line of an element differently. You can also use pseudo-elements to add new content before or after the selected element.

In contrast to pseudo-classes that are preceded by a `:` character, pseudo-elements are preceded by a `::` character.

The `::` character is followed by a keyword that allows you to style a specific part of the selected element.

The general syntax looks something like the following:

```
element::pseudo-element-selector {
    property:value;
}
```

Make sure to use the `::` character instead of the `:` one when using pseudo-element selectors - this will help distinguish pseudo-classes from pseudo-elements.

Now, let's see some of the most common pseudo-elements you will encounter.

The `::before` Pseudo-Element

You can use the `::before` pseudo-element to insert content before an element:

```
p::before {
    property: value;
}
```

The `::after` Pseudo-Element

And you can use the `::after` pseudo-element to insert content at the end of an element:

```
p::after {
    property: value;
}
```

The `::first-letter` Pseudo-Element

You can also use the `::first-letter` pseudo-element to select the first letter of a paragraph, which is helpful when you want to style the first letter in a certain way:

```
p::first-letter {
    property: value;
}
```

The `::first-line` Pseudo-Element

And you can use the `::first-line` pseudo-element to select the first line of a paragraph:

```
p::first-line {
    property: value;
```

```
}
```

Conclusion

Hopefully you found this cheat sheet of the most widely used CSS selectors helpful.

To learn more about web design, check out freeCodeCamp's Responsive Web Design Certification. In the interactive lessons, you'll learn HTML and CSS by building 15 practice projects and 5 certification projects.

Thanks for reading, and happy coding!

Dionysia Lemonaki

Read more posts.
